

1 A simple imperative language

We will now consider a more realistic programming language, one where we can assign values to variables and execute control constructs such as if and while. The syntax for this imperative language, called IMP, is as follows:

arithmetic expressions $a \in \mathbf{Aexp}$ $a ::= x \mid n \mid a_1 + a_2 \mid a_1 \times a_2$
 Boolean expressions $b \in \mathbf{Bexp}$ $b ::= \mathbf{true} \mid \mathbf{false} \mid a_1 < a_2$
 commands $c \in \mathbf{Com}$ $c ::= \mathbf{skip} \mid x := a \mid c_1; c_2 \mid \mathbf{if } b \mathbf{ then } c_1 \mathbf{ else } c_2 \mid \mathbf{while } b \mathbf{ do } c$

1.1 Small-step operational semantics

We'll first give a small-step operational semantics for IMP. The configurations in this language are of the form $\langle \sigma, c \rangle$, $\langle \sigma, b \rangle$, and $\langle \sigma, a \rangle$, where σ is a store. The final configurations are of the form $\langle \sigma, \mathbf{skip} \rangle$ for commands, $\langle \sigma, \mathbf{true} \rangle$ and $\langle \sigma, \mathbf{false} \rangle$ for Boolean expressions, and $\langle \sigma, n \rangle$ for arithmetic expressions. There are three different small-step operational semantics relations: one each of the syntactic categories.

$$\begin{aligned}\rightarrow_{\mathbf{Com}} &\subseteq (\mathbf{Store} \times \mathbf{Com}) \times (\mathbf{Store} \times \mathbf{Com}) \\ \rightarrow_{\mathbf{Bexp}} &\subseteq (\mathbf{Store} \times \mathbf{Bexp}) \times (\mathbf{Store} \times \mathbf{Bexp}) \\ \rightarrow_{\mathbf{Aexp}} &\subseteq (\mathbf{Store} \times \mathbf{Aexp}) \times (\mathbf{Store} \times \mathbf{Aexp})\end{aligned}$$

For brevity, we will overload the symbol $\rightarrow$ and use it to refer to all of these relations. Which relation is being used will be clear from context. The evaluation rules for arithmetic and Boolean expressions are similar to the ones we've seen before. However, note that since the arithmetic expressions no longer contain assignment, arithmetic and Boolean expressions can not update the store.

Arithmetic expressions

$$\begin{array}{c} \frac{n = \sigma(x)}{\langle \sigma, x \rangle \rightarrow \langle \sigma, n \rangle} \\[10pt] \frac{\langle \sigma, a_1 \rangle \rightarrow \langle \sigma, a'_1 \rangle}{\langle \sigma, a_1 + a_2 \rangle \rightarrow \langle \sigma, a'_1 + a_2 \rangle} \quad \frac{\langle \sigma, a_2 \rangle \rightarrow \langle \sigma, a'_2 \rangle}{\langle \sigma, n + a_2 \rangle \rightarrow \langle \sigma, n + a'_2 \rangle} \quad \frac{p = n + m}{\langle \sigma, n + m \rangle \rightarrow \langle \sigma, p \rangle} \\[10pt] \frac{\langle \sigma, a_1 \rangle \rightarrow \langle \sigma, a'_1 \rangle}{\langle \sigma, a_1 \times a_2 \rangle \rightarrow \langle \sigma, a'_1 \times a_2 \rangle} \quad \frac{\langle \sigma, a_2 \rangle \rightarrow \langle \sigma, a'_2 \rangle}{\langle \sigma, n \times a_2 \rangle \rightarrow \langle \sigma, n \times a'_2 \rangle} \quad \frac{p = n \times m}{\langle \sigma, n \times m \rangle \rightarrow \langle \sigma, p \rangle}\end{array}$$

Boolean expressions

$$\frac{\langle \sigma, a_1 \rangle \rightarrow \langle \sigma, a'_1 \rangle}{\langle \sigma, a_1 < a_2 \rangle \rightarrow \langle \sigma, a'_1 < a_2 \rangle}$$

$$\frac{\langle \sigma, a_2 \rangle \rightarrow \langle \sigma, a'_2 \rangle}{\langle \sigma, n < a_2 \rangle \rightarrow \langle \sigma, n < a'_2 \rangle}$$

$$\frac{n < m}{\langle \sigma, n < m \rangle \rightarrow \langle \sigma, \mathbf{true} \rangle}$$

$$\frac{n \geq m}{\langle \sigma, n < m \rangle \rightarrow \langle \sigma, \mathbf{false} \rangle}$$

Commands

$$\frac{\langle \sigma, a \rangle \rightarrow \langle \sigma, a' \rangle}{\langle \sigma, x := a \rangle \rightarrow \langle \sigma, x := a' \rangle}$$

$$\frac{}{\langle \sigma, x := n \rangle \rightarrow \langle \sigma[x \mapsto n], \mathbf{skip} \rangle}$$

$$\frac{\langle \sigma, c_1 \rangle \rightarrow \langle \sigma', c'_1 \rangle}{\langle \sigma, c_1; c_2 \rangle \rightarrow \langle \sigma', c'_1; c_2 \rangle}$$

$$\frac{}{\langle \sigma, \mathbf{skip}; c_2 \rangle \rightarrow \langle \sigma, c_2 \rangle}$$

For if commands, we reduce the test until we get **true** or **false** and then we execute the appropriate branch:

$$\frac{\langle \sigma, b \rangle \rightarrow \langle \sigma, b' \rangle}{\langle \sigma, \mathbf{if } b \mathbf{ then } c_1 \mathbf{ else } c_2 \rangle \rightarrow \langle \sigma, \mathbf{if } b' \mathbf{ then } c_1 \mathbf{ else } c_2 \rangle}$$

$$\frac{}{\langle \sigma, \mathbf{if true then } c_1 \mathbf{ else } c_2 \rangle \rightarrow \langle \sigma, c_1 \rangle}$$

$$\frac{}{\langle \sigma, \mathbf{if false then } c_1 \mathbf{ else } c_2 \rangle \rightarrow \langle \sigma, c_2 \rangle}$$

For while loops, the above strategy doesn't work (why?). Instead, we use the following rule, which can be thought of as "unrolling" the loop, one iteration at a time.

$$\frac{}{\langle \sigma, \mathbf{while } b \mathbf{ do } c \rangle \rightarrow \langle \sigma, \mathbf{if } b \mathbf{ then } (c; \mathbf{while } b \mathbf{ do } c) \mathbf{ else skip} \rangle}$$

We can now take a concrete program and see how it executes under the above rules. Consider we execute the program

foo := 3; **while** foo < 4 **do** foo := foo + 5

The execution works as follows:

$$\begin{aligned}
& \langle \sigma, \text{foo} := 3; \text{while } \text{foo} < 4 \text{ do } \text{foo} := \text{foo} + 5 \rangle \\
& \rightarrow \langle \sigma', \text{skip}; \text{while } \text{foo} < 4 \text{ do } \text{foo} := \text{foo} + 5 \rangle & \text{where } \sigma' = \sigma[\text{foo} \mapsto 3] \\
& \rightarrow \langle \sigma', \text{while } \text{foo} < 4 \text{ do } \text{foo} := \text{foo} + 5 \rangle \\
& \rightarrow \langle \sigma', \text{if } \text{foo} < 4 \text{ then } (\text{foo} := \text{foo} + 5; W) \text{ else skip} \rangle \\
& \rightarrow \langle \sigma', \text{if } 3 < 4 \text{ then } (\text{foo} := \text{foo} + 5; W) \text{ else skip} \rangle \\
& \rightarrow \langle \sigma', \text{if true then } (\text{foo} := \text{foo} + 5; W) \text{ else skip} \rangle \\
& \rightarrow \langle \sigma', \text{foo} := \text{foo} + 5; \text{while } \text{foo} < 4 \text{ do } \text{foo} := \text{foo} + 5 \rangle \\
& \rightarrow \langle \sigma', \text{foo} := 3 + 5; \text{while } \text{foo} < 4 \text{ do } \text{foo} := \text{foo} + 5 \rangle \\
& \rightarrow \langle \sigma', \text{foo} := 8; \text{while } \text{foo} < 4 \text{ do } \text{foo} := \text{foo} + 5 \rangle \\
& \rightarrow \langle \sigma'', \text{skip}; \text{while } \text{foo} < 4 \text{ do } \text{foo} := \text{foo} + 5 \rangle & \text{where } \sigma'' = \sigma'[\text{foo} \mapsto 8] \\
& \rightarrow \langle \sigma'', \text{while } \text{foo} < 4 \text{ do } \text{foo} := \text{foo} + 5 \rangle \\
& \rightarrow \langle \sigma'', \text{if } \text{foo} < 4 \text{ then } (\text{foo} := \text{foo} + 5; W) \text{ else skip} \rangle \\
& \rightarrow \langle \sigma'', \text{if } 8 < 4 \text{ then } (\text{foo} := \text{foo} + 5; W) \text{ else skip} \rangle \\
& \rightarrow \langle \sigma'', \text{if false then } (\text{foo} := \text{foo} + 5; W) \text{ else skip} \rangle \\
& \rightarrow \langle \sigma'', \text{skip} \rangle
\end{aligned}$$

where W is an abbreviation for the while loop $\text{while } \text{foo} < 4 \text{ do } \text{foo} := \text{foo} + 5$.

2 Large-step operational semantics for IMP

We define large-step evaluation relations for arithmetic expressions, Boolean expressions, and commands. The relation for arithmetic expressions relates an arithmetic expression and store to the integer value that the expression evaluates to. For Boolean expressions, the final value is in $\text{Bool} = \{\text{true}, \text{false}\}$. For commands, the final value is a store.

$$\begin{aligned}
\Downarrow_{\text{Aexp}} &\subseteq (\text{Aexp} \times \text{Store}) \times \text{Int} \\
\Downarrow_{\text{Bexp}} &\subseteq (\text{Bexp} \times \text{Store}) \times \text{Bool} \\
\Downarrow_{\text{Com}} &\subseteq (\text{Com} \times \text{Store}) \times \text{Store}
\end{aligned}$$

Again, we overload the symbol $\Downarrow$ and use it for any of these three relations; which relation is intended will be clear from context. We also use infix notation, for example writing $\langle \sigma, c \rangle \Downarrow \sigma'$ if $(\langle \sigma, c \rangle, \sigma') \in \Downarrow_{\text{Com}}$.

Arithmetic expressions.

$$\begin{array}{c}
\frac{}{\langle \sigma, n \rangle \Downarrow n} \qquad \frac{\sigma(x) = n}{\langle \sigma, x \rangle \Downarrow n} \\
\\
\frac{\langle \sigma, a_1 \rangle \Downarrow n_1 \quad \langle \sigma, a_2 \rangle \Downarrow n_2 \quad n = n_1 + n_2}{\langle \sigma, a_1 + a_2 \rangle \Downarrow n} \qquad \frac{\langle \sigma, a_1 \rangle \Downarrow n_1 \quad \langle \sigma, a_2 \rangle \Downarrow n_2 \quad n = n_1 \times n_2}{\langle \sigma, a_1 \times a_2 \rangle \Downarrow n}
\end{array}$$

Boolean expressions.

$$\begin{array}{c}
\frac{}{\langle \sigma, \mathbf{true} \rangle \Downarrow \mathbf{true}} \qquad \frac{}{\langle \sigma, \mathbf{false} \rangle \Downarrow \mathbf{false}} \\
\\
\frac{\langle \sigma, a_1 \rangle \Downarrow n_1 \quad \langle \sigma, a_2 \rangle \Downarrow n_2 \quad n_1 < n_2}{\langle \sigma, a_1 < a_2 \rangle \Downarrow \mathbf{true}} \qquad \frac{\langle \sigma, a_1 \rangle \Downarrow n_1 \quad \langle \sigma, a_2 \rangle \Downarrow n_2 \quad n_1 \geq n_2}{\langle \sigma, a_1 < a_2 \rangle \Downarrow \mathbf{false}}
\end{array}$$

Commands.

$$\begin{array}{c}
\text{SKIP} \frac{}{\langle \sigma, \mathbf{skip} \rangle \Downarrow \sigma} \quad \text{ASSGN} \frac{\langle \sigma, a \rangle \Downarrow n}{\langle \sigma, x := a \rangle \Downarrow \sigma[x \mapsto n]} \quad \text{SEQ} \frac{\langle \sigma, c_1 \rangle \Downarrow \sigma' \quad \langle \sigma', c_2 \rangle \Downarrow \sigma''}{\langle \sigma, c_1; c_2 \rangle \Downarrow \sigma''} \\
\\
\text{IF-T} \frac{\langle \sigma, b \rangle \Downarrow \mathbf{true} \quad \langle \sigma, c_1 \rangle \Downarrow \sigma'}{\langle \sigma, \mathbf{if } b \mathbf{ then } c_1 \mathbf{ else } c_2 \rangle \Downarrow \sigma'} \quad \text{IF-F} \frac{\langle \sigma, b \rangle \Downarrow \mathbf{false} \quad \langle \sigma, c_2 \rangle \Downarrow \sigma'}{\langle \sigma, \mathbf{if } b \mathbf{ then } c_1 \mathbf{ else } c_2 \rangle \Downarrow \sigma'} \\
\\
\text{WHILE-F} \frac{\langle \sigma, b \rangle \Downarrow \mathbf{false}}{\langle \sigma, \mathbf{while } b \mathbf{ do } c \rangle \Downarrow \sigma} \quad \text{WHILE-T} \frac{\langle \sigma, b \rangle \Downarrow \mathbf{true} \quad \langle \sigma, c \rangle \Downarrow \sigma' \quad \langle \sigma', \mathbf{while } b \mathbf{ do } c \rangle \Downarrow \sigma''}{\langle \sigma, \mathbf{while } b \mathbf{ do } c \rangle \Downarrow \sigma''}
\end{array}$$

It's interesting to see that the rule for while loops does not rely on using an if command (as in the case of small-step semantics). Why does this rule work?

2.1 Command equivalence

The small-step operational semantics suggests that the loop **while** b **do** c should be equivalent to the command **if** b **then** $(c; \mathbf{while } b \mathbf{ do } c)$ **else skip**. Can we show that this indeed the case that the language is defined using the above large-step evaluation?

First, we need to be more precise about what “equivalent commands” mean. Our formal model allows us to define this concept using large-step evaluations as follows. (One can write a similar definition using $\rightarrow^*$ in small-step semantics.)

Definition (Equivalence of commands). Two commands c and c' are equivalent (written $c \sim c'$) if, for any stores σ and σ' , we have

$$\langle \sigma, c \rangle \Downarrow \sigma' \iff \langle \sigma, c' \rangle \Downarrow \sigma'.$$

We can now state and prove the claim that **while** b **do** c and **if** b **then** $(c; \mathbf{while } b \mathbf{ do } c)$ **else skip** are equivalent.

Theorem. For all $b \in \mathbf{Bexp}$ and $c \in \mathbf{Com}$ we have

$$\mathbf{while } b \mathbf{ do } c \sim \mathbf{if } b \mathbf{ then } (c; \mathbf{while } b \mathbf{ do } c) \mathbf{ else skip}.$$

Proof. Let W be an abbreviation for **while** b **do** c . We want to show that for all stores σ, σ' , we have:

$$\langle \sigma, W \rangle \Downarrow \sigma' \text{ if and only if } \langle \sigma, \mathbf{if } b \mathbf{ then } (c; W) \mathbf{ else skip} \rangle \Downarrow \sigma'$$

For this, we must show that both directions ($\implies$ and $\impliedby$) hold. We'll show only direction $\implies$; the other is similar.

Assume that σ and σ' are stores such that $\langle \sigma, W \rangle \Downarrow \sigma'$. It means that there is some derivation that proves for this fact. Inspecting the evaluation rules, we see that there are two possible rules whose conclusions match this fact: **WHILE-F** and **WHILE-T**. We analyze each of them in turn.

- **WHILE-F**. The derivation must look like the following.

$$\text{WHILE-F} \frac{\frac{\vdots^1}{\langle \sigma, b \rangle \Downarrow \text{false}}}{\langle \sigma, W \rangle \Downarrow \sigma}$$

Here, we use $\vdots^1$ to refer to the derivation of $\langle \sigma, b \rangle \Downarrow \text{false}$. Note that in this case, $\sigma' = \sigma$.

We can use $\vdots^1$ to derive a proof tree showing that the evaluation of **if b then $(c; W)$ else skip** yields the same final state σ :

$$\text{IF-F} \frac{\frac{\vdots^1}{\langle \sigma, b \rangle \Downarrow \text{false}} \quad \text{SKIP} \frac{}{\langle \sigma, \text{skip} \rangle \Downarrow \sigma}}{\langle \sigma, \text{if } b \text{ then } (c; W) \text{ else skip} \rangle \Downarrow \sigma}$$

- **WHILE-T**. In this case, the derivation has the following form.

$$\text{WHILE-T} \frac{\frac{\vdots^2}{\langle \sigma, b \rangle \Downarrow \text{true}} \quad \frac{\vdots^3}{\langle \sigma, c \rangle \Downarrow \sigma''} \quad \frac{\vdots^4}{\langle \sigma'', W \rangle \Downarrow \sigma'}}{\langle \sigma, W \rangle \Downarrow \sigma'}$$

We can use subderivations $\vdots^2$, $\vdots^3$, and $\vdots^4$ to show that the evaluation of **if b then $(c; W)$ else skip** yields the same final state σ .

$$\text{IF-T} \frac{\frac{\vdots^2}{\langle \sigma, b \rangle \Downarrow \text{true}} \quad \text{SEQ} \frac{\frac{\vdots^3}{\langle \sigma, c \rangle \Downarrow \sigma''} \quad \frac{\vdots^4}{\langle \sigma'', W \rangle \Downarrow \sigma'}}{\langle \sigma, c; W \rangle \Downarrow \sigma'}}{\langle \sigma, \text{if } b \text{ then } (c; W) \text{ else skip} \rangle \Downarrow \sigma'}$$

Hence, we showed that in each of the two possible cases, the command **if b then $(c; W)$ else skip** evaluates to the same final state as the command W . $\square$